

Consul Service Mesh

Department: DevOps

Prepared by: Thanusha Bai V

Date: 30-sept-2026

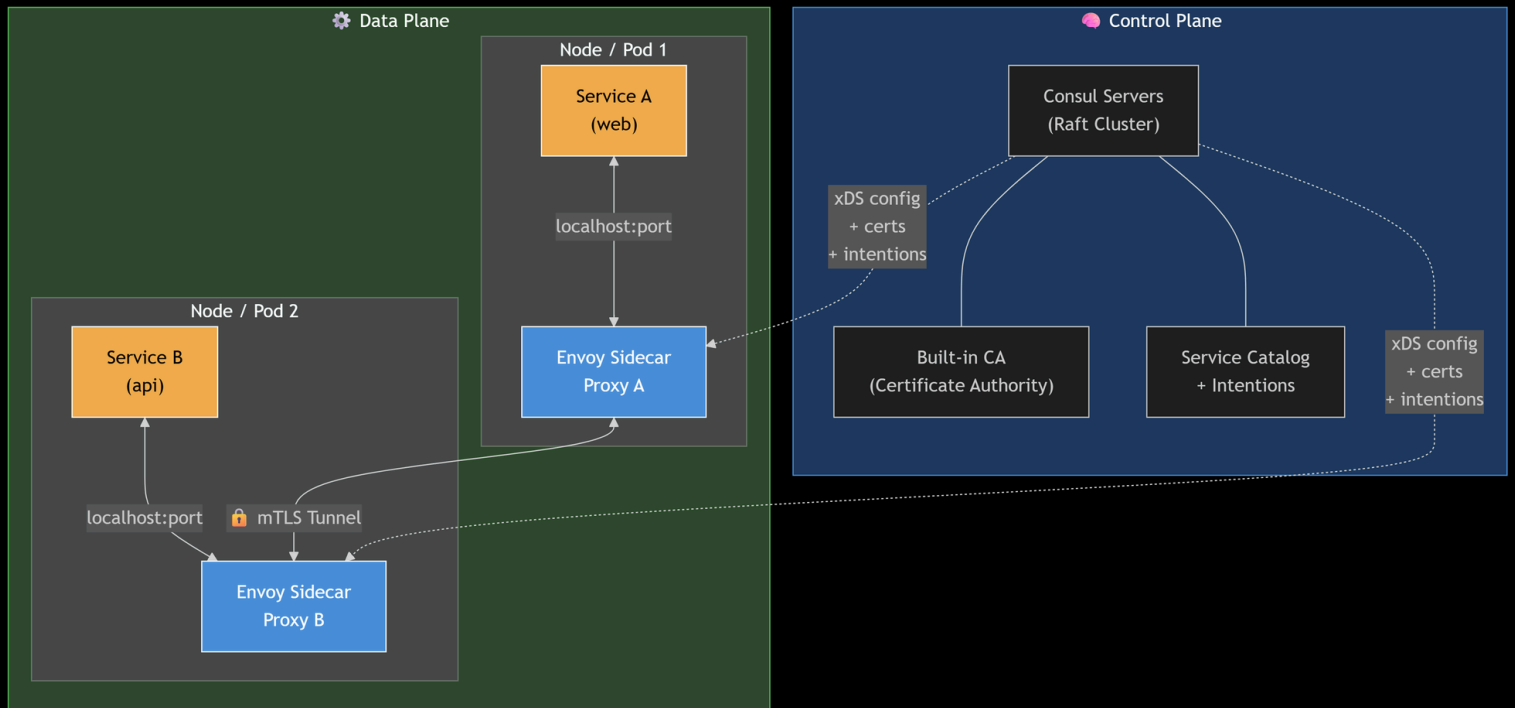
Subtask: DEV-611-Research Consul Connect architecture

1. Overview

Consul Service Mesh (powered by the Connect subsystem) is a solution for authorizing and encrypting service-to-service communication. Its core goal is to provide automatic mTLS encryption, identity-based access control, and unified traffic observability – without requiring any changes to application code.

The architecture follows the classic control plane + data plane separation model:

- **Control plane** – Consul handles service discovery, certificate issuance, and intention management.
- **Data plane** – Proxies (sidecars) handle actual traffic forwarding, TLS termination, and authorization enforcement.



High-Level (Control Plane + Data Plane)

2. Core Components

Consul Service Mesh is built around four core components.

2.1 Control Plane (Consul Servers)

The Consul server cluster is the "brain" of the mesh and is responsible for:

- **Service catalog management** – maintains service registrations, health status, and endpoint addresses

- **Intention management** – stores and distributes service-to-service authorization rules
- **Certificate issuance** – the built-in CA issues TLS certificates to every service instance
- **Proxy configuration distribution** – pushes dynamic config to Envoy proxies via the xDS API

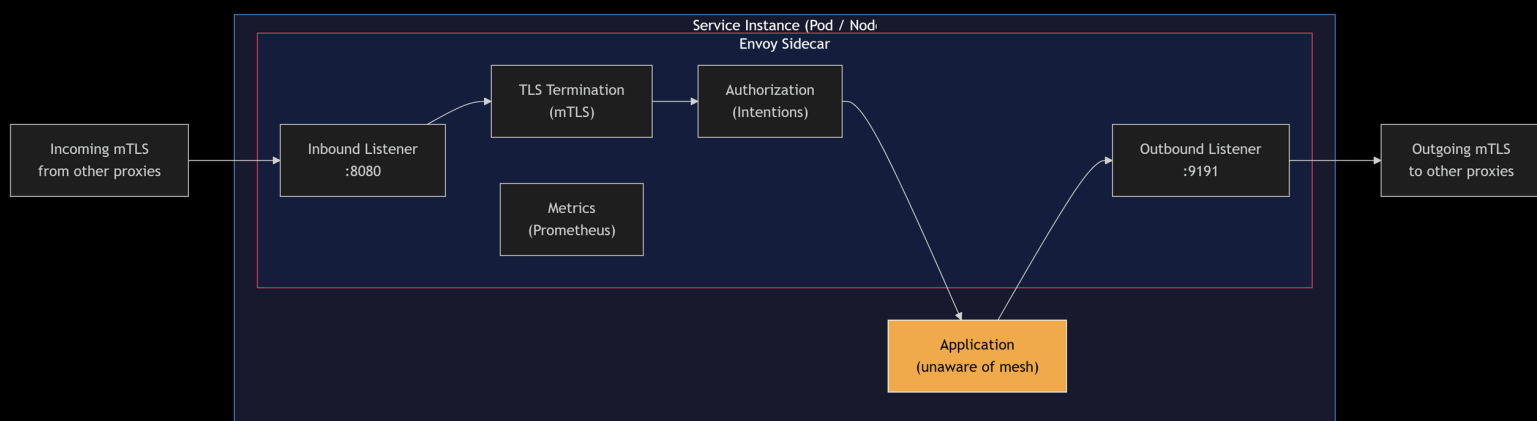
2.2 Data Plane (Sidecar Proxy)

A lightweight proxy (Envoy by default) runs alongside every service instance and intercepts all inbound/outbound traffic:

- **Inbound traffic** – terminates mTLS connections, verifies client certificates, checks intention rules, and forwards plaintext requests to the local service
- **Outbound traffic** – initiates mTLS connections to the target service's proxy, handling encryption and certificate verification

- **Observability** – collects L7 metrics (e.g., HTTP status codes, latency) that can be exported to Prometheus

Note: Envoy is the recommended production proxy and supports both L4 and L7 features. Consul also offers a built-in proxy for dev/testing, but it is limited in functionality and not recommended for production.

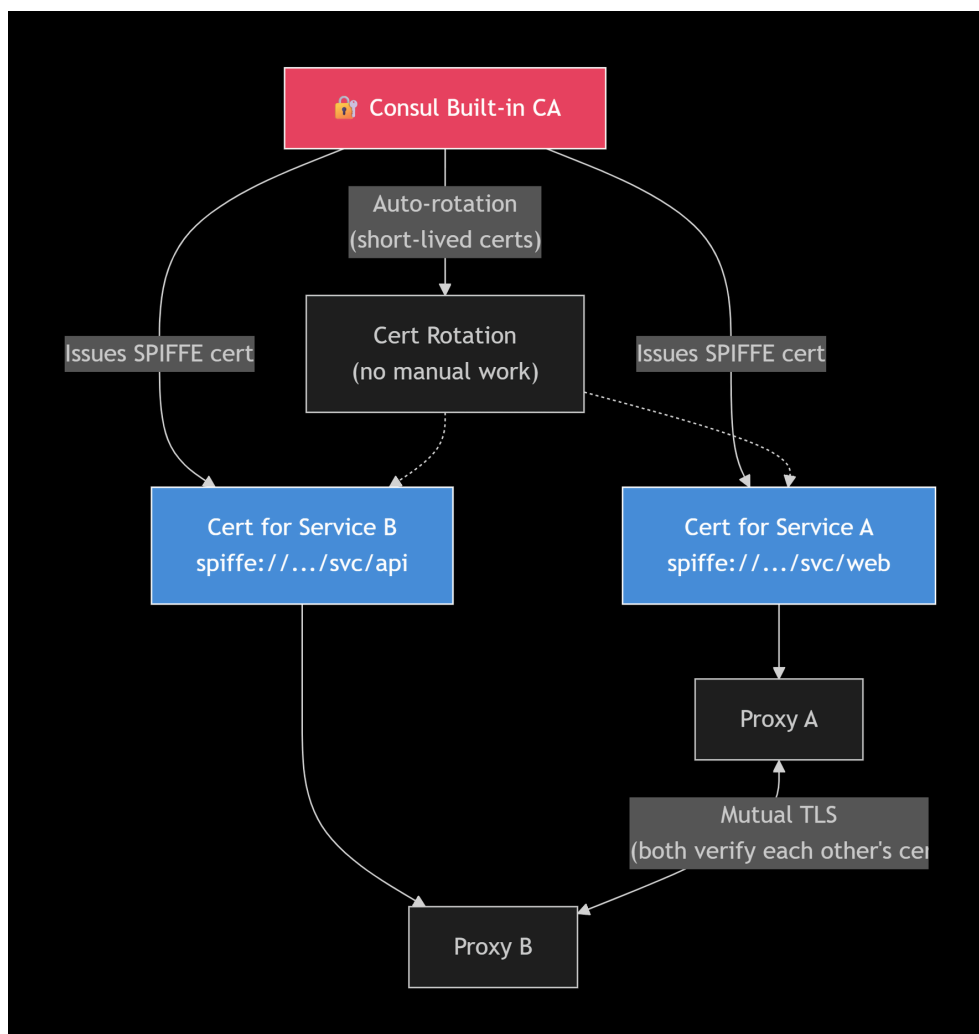


Sidecar Proxy (Inbound & Outbound)

2.3 Certificate Authority (CA)

Consul's built-in CA (or a delegated Vault / AWS Private CA) issues short-lived TLS certificates to every service instance:

- **Identity** – certificates use the SPIFFE format, e.g. `spiffe://trust-domain/ns/namespace/dc/datacenter/svc/service-name`
- **Automatic rotation** – certificates are renewed and rotated automatically, with no manual intervention
- **mTLS foundation** – client and server mutually verify certificates, achieving two-way identity authentication

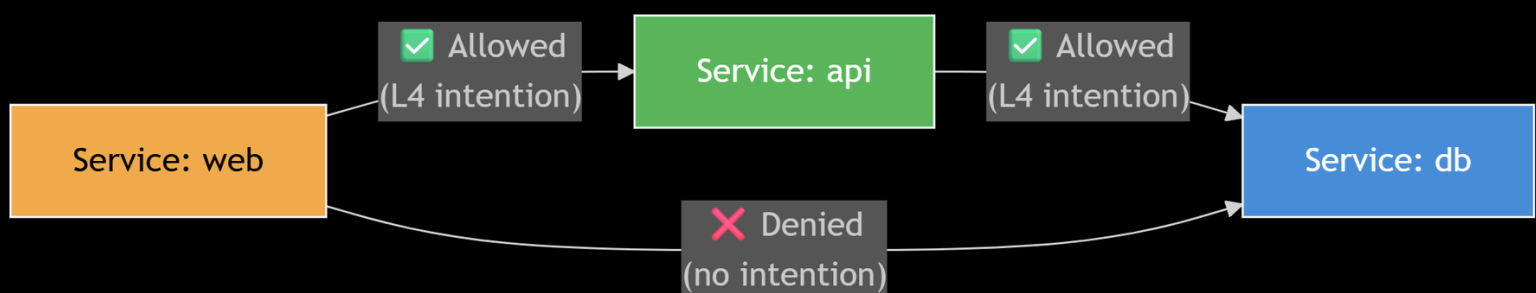


Certificate & Identity Flow

2.4 Intentions

Intentions are authorization rules for service-to-service communication, enforced by the data plane proxies:

Type	Layer	Capability
L4 intention	Connection-level	Allow/deny based on service identity
L7 intention	Request-level	Path, header, HTTP method filtering (Envoy only)



Intention Enforcement

3. Workflow

3.1 Service Registration and Proxy Configuration

A service is registered to Consul together with its sidecar proxy. A typical registration config:

```
service {  
  name = "web"  
  port = 8080  
    
  connect {  
    sidecar_service {  
      proxy {  
        upstreams = [  
          { destination_name = "api", local_bind_port = 9191 }  
        ]  
      }  
    }  
  }  
}
```

Key Fields Explained

Field	Purpose
<code>connect.sidecar_service {}</code>	Declares that a sidecar proxy is registered for this service
<code>upstreams</code>	Declares the upstream services this service needs to reach
<code>local_bind_port</code>	The app uses this local port to reach the upstream; the proxy handles the actual connection setup

After registration, the proxy process is started separately:

```
consul connect envoy -sidecar-for web
```

3.2 Traffic Flow

When application A calls service B, the traffic goes through these steps:

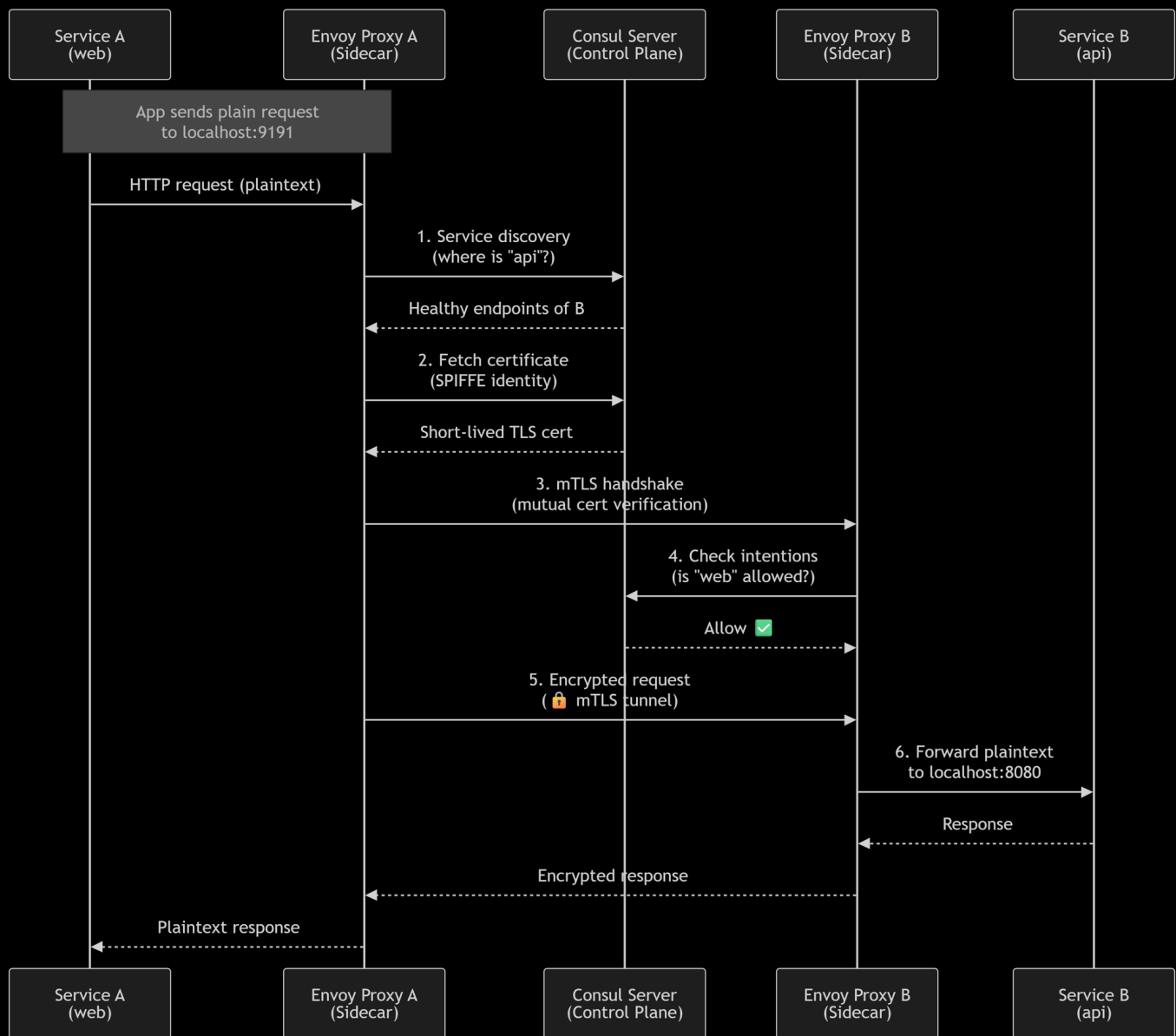
1. **Service discovery** – proxy A queries Consul for healthy endpoints of service B.
2. **mTLS handshake** – proxy A and proxy B complete a mutual TLS handshake

using certificates issued by Consul.

3. **Authorization check** – proxy B checks the intention rules fetched from Consul to confirm A is allowed to connect.
4. **Proxy-to-proxy communication** – encrypted traffic flows between the two Envoy proxies.
5. **Local forwarding** – proxy B forwards the decrypted request to the local service B.

Note: Application A only needs to send plain requests to localhost:

<local_bind_port> – all encryption and routing logic is transparent to the application.



Traffic Flow (Sequence)

4. Key Design Decisions

4.1 Protocol-Agnostic Core

Consul Service Mesh's core is built around L4 (transport layer), not bound to L7 protocols like HTTP/gRPC.

Rationale:

- **Compatibility first** – many legacy systems (databases, mainframes) don't support gRPC or HTTP semantics
- **Performance** – L4 authorization runs once per TCP connection, not per HTTP request, so overhead is lower
- **Progressive enhancement** – baseline security doesn't depend on L7 features; L7 capabilities are added via Envoy integration

4.2 Pluggable Data Plane

Consul does not force a specific proxy. While Envoy is the default and recommended choice, the architecture supports third-party proxy integration (e.g., HAProxy, Nginx).

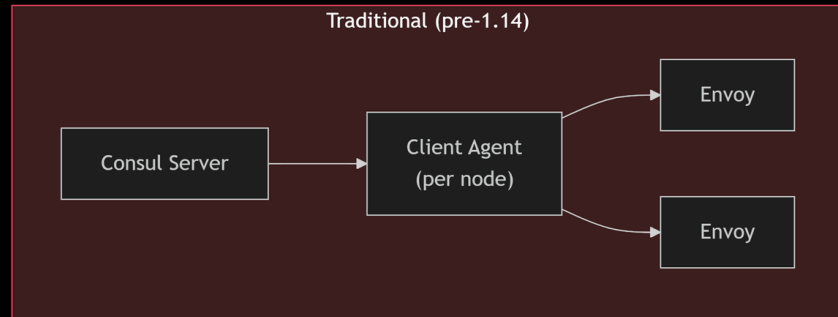
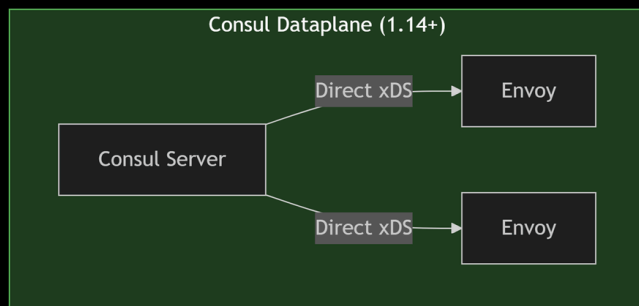
Proxy	Best suited for
Envoy	Performance-critical apps, L7 features needed
Built-in proxy	Dev/test, non-performance-critical L4 apps
HAProxy / others	Teams with existing stack and operational experience

4.3 Consul Dataplane (v1.14+)

Consul 1.14 introduced Consul Dataplane, which removes the dependency on node-level client agents. This change directly affects performance testing:

- **Lower resource usage** – smaller deployments consume fewer overall resources
- **Control plane load shift** – Consul servers must generate xDS resources; resource consumption shifts to the server side

- **Linear scaling** – as the number of proxies grows, dataplane resource usage scales linearly



Traditional vs. Consul Dataplane

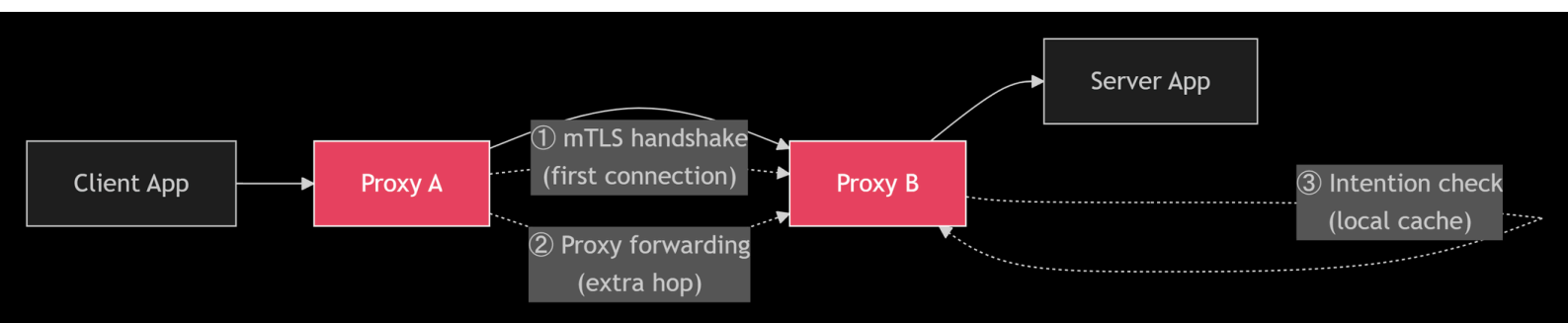
5. Takeaways for This Task's Performance Testing

Based on the architecture analysis, the upcoming subtasks should focus on the following.

5.1 Sources of Latency Difference

Overhead mainly comes from three areas:

Area	Source of overhead	What to test
mTLS handshake	Certificate exchange and verification	First-connection latency vs. long-lived connection reuse
Proxy forwarding	User-space ↔ kernel-space context switches	Impact of extra network hops
Authorization check	Intention rule matching	Impact of local cache hit ratio



Latency Overhead Sources

5.2 CPU / Memory Overhead

- **Envoy process** – one proxy process per service instance; measure per-instance resource usage
- **Certificate rotation** – impact of automatic TLS cert rotation on steady-state performance

- **Metrics collection** – extra CPU cost of L7 observability features

5.3 Hypotheses Worth Validating

- First-connection latency is significantly higher than subsequent requests (mTLS handshake cost)
 - In long-lived connection scenarios, the steady-state latency added by the proxy is small
 - Under Consul Dataplane mode, client-side resource usage drops but server-side load increases
 - L4 intentions have negligible performance impact (local cache, microsecond-level response)
-

6. Link to Subsequent Subtasks

This document provides the theoretical basis for:

Subtask	What it builds on from this doc
DEV-612	Build the "with mesh" vs. "without mesh" comparison environment based on Section 3.1
DEV-613/614	Design latency and resource measurement plans based on the overhead sources in Section 5
DEV-615	Test traffic control and failover behavior based on the intention mechanism
DEV-616	Write up the security benefits doc based on Sections 2.3 and 2.4

Conclusion

This document establishes the theoretical foundation for evaluating Consul Service Mesh, with particular focus on its control plane, data plane, mTLS security, intention-based authorization, proxy architecture, and Consul Dataplane.

The identified latency, CPU, memory, certificate rotation, authorization, and observability overheads provide the basis for the subsequent performance testing subtasks.